

Relazione di Progetto

Sistema Informativo per Noleggio Barche

Introduzione e Motivazione della Scelta

Il presente testo ripercorre le principali fasi di **analisi, progettazione e sviluppo** che hanno portato alla realizzazione del **Sistema Informativo per la Gestione del Noleggio di Barche**, sviluppato nell'ambito del progetto di Programmazione Object Oriented. Il lavoro è stato organizzato seguendo i **tre homework** previsti, in modo da passare gradualmente dalla definizione del dominio alla prima versione applicativa e, infine, al completamento del progetto e della documentazione.

La scelta del dominio è ricaduta sulla gestione di un servizio di noleggio di barche perché permette di rappresentare un contesto reale composto da più elementi tra loro collegati. Il sistema deve infatti coordinare **clienti, imbarcazioni, sedi, prenotazioni, noleggi e manutenzioni**, tenendo conto anche di vincoli concreti come la **capienza delle barche**, la **patente nautica**, la **disponibilità nel periodo richiesto** e gli **stati operativi** delle imbarcazioni.

Durante l'**analisi preliminare** ho cercato di individuare gli elementi realmente utili al funzionamento del sistema, evitando di inserire informazioni non necessarie. Il dominio è stato quindi costruito attorno alle operazioni principali del servizio: **registrazione del cliente, consultazione delle barche disponibili, creazione di una prenotazione, avvio e conclusione del noleggio e gestione delle manutenzioni** da parte dell'amministratore.

Questa prima analisi ha costituito il punto di partenza per definire le **entità**, le **relazioni** e le **regole del progetto**. A partire da tali scelte è stato poi possibile costruire il **modello Object Oriented**, organizzare l'**architettura applicativa** e realizzare un'**interfaccia grafica** collegata alla logica del sistema e alla persistenza dei dati.

Analisi iniziale e progettazione del dominio

Nel corso del primo homework il lavoro si è concentrato sulla **fase iniziale di analisi e progettazione del dominio**. In questa fase non mi sono ancora occupato dell'**implementazione del programma**, ma ho definito che cosa il sistema dovesse rappresentare, quali fossero le **entità principali** e quali **regole** dovessero essere rispettate durante le successive fasi di sviluppo.

Il dominio scelto riguarda un servizio organizzato di **noleggio di barche** gestito attraverso più sedi operative. Ogni sede può disporre di una propria flotta e il sistema deve permettere di distinguere le imbarcazioni disponibili da quelle già noleggiate, in manutenzione oppure fuori servizio. Dal punto di vista dell'utente, il cliente deve poter consultare le barche e prenotarne una per un determinato periodo, mentre l'amministratore deve poter gestire le operazioni successive di ritiro, restituzione e manutenzione.

A partire da questa analisi sono state individuate le classi principali del dominio: **Cliente**, **Barca**, **BarcaMotore**, **BarcaVela**, **Sede**, **Prenotazione**, **Noleggio** e **Manutenzione**. La classe **Barca** è stata progettata come classe astratta, in modo da raccogliere le informazioni comuni a tutte le imbarcazioni e lasciare alle sottoclassi le caratteristiche specifiche. In particolare, **BarcaMotore** contiene i dati legati alla potenza del motore e alla capacità del serbatoio, mentre **BarcaVela** utilizza la superficie velica e l'altezza dell'albero.

La classe **Cliente** rappresenta l'utente che utilizza il servizio e conserva anche le informazioni relative all'eventuale patente nautica. La **Prenotazione** collega il cliente alla barca scelta e definisce il periodo richiesto e il numero di passeggeri. Il **Noleggio** rappresenta invece l'utilizzo effettivo della barca dopo il ritiro, mentre la **Manutenzione** permette di gestire i periodi nei quali un'imbarcazione non può essere utilizzata. Infine, la **Sede** identifica il punto operativo al quale appartiene ciascuna barca.

Parallelamente alle classi sono stati definiti anche gli **stati principali** del sistema. La barca può essere disponibile, noleggiata, in manutenzione o fuori servizio; la prenotazione può essere confermata, noleggiata, completata o annullata; il noleggio può essere in corso, sospeso o terminato; la manutenzione può essere programmata, in corso o completata. L'uso di questi stati permette di rappresentare in modo chiaro l'evoluzione degli oggetti durante il funzionamento dell'applicazione.

Durante la progettazione sono state inoltre individuate le **regole di dominio e di business** più importanti. Tra queste rientrano il controllo della maggiore età del cliente, l'obbligo di una patente nautica valida quando la barca la richiede, il rispetto della capacità massima dei passeggeri, la validità del periodo richiesto e il controllo delle sovrapposizioni tra prenotazioni e indisponibilità della barca. Sono state previste anche eccezioni specifiche per rappresentare in modo chiaro i principali casi di errore.

Sulla base delle entità e delle regole individuate è stato realizzato il **Class Diagram UML** del dominio. Il diagramma ha rappresentato graficamente le classi, gli attributi, i metodi principali, le enumerazioni e le relazioni tra gli oggetti. In particolare, la generalizzazione tra **Barca**, **BarcaMotore** e **BarcaVela** è diventata uno degli elementi centrali del modello Object Oriented.

Il primo homework si è quindi concluso con una **prima documentazione del progetto**, contenente la presentazione del dominio, la descrizione del sistema, il diagramma delle classi e il riferimento alla **repository GitHub**. Questa fase ha costituito la base progettuale utilizzata successivamente per trasformare il modello concettuale in **classi Java** e in un'applicazione funzionante.

Sviluppo della prima versione applicativa

Nel corso del secondo homework il progetto è passato dalla progettazione alla realizzazione di una **prima versione concreta dell'applicazione**. Il modello definito nel primo homework è stato trasformato in **classi Java** e sono stati introdotti i componenti necessari per collegare il **dominio**, la **logica applicativa**, la **persistenza dei dati** e l'**interfaccia grafica**.

Prima dell'implementazione è stato riesaminato il **Class Diagram** per verificare che classi, responsabilità e relazioni fossero coerenti con il comportamento reale del sistema. Questo controllo ha permesso di consolidare il ruolo delle principali entità e di rendere più chiari i collegamenti tra Cliente, Barca, Prenotazione, Noleggio, Manutenzione e Sede. Il diagramma è quindi diventato il riferimento per la creazione del package **model** e delle classi effettivamente utilizzate dal programma.

Il codice è stato organizzato seguendo l'architettura **BCE + DAO**. Il livello **Entity** è rappresentato dalle classi del **model**; il livello **Control** è affidato al **NoleggioBarcheController**; il livello **Boundary** è costituito dai pannelli della GUI realizzati con Java Swing. A questa struttura è stato aggiunto il livello **DAO**, utilizzato per separare l'accesso ai dati dalla logica applicativa.

Nel **model** sono state implementate le entità progettate nel primo homework, le enumerazioni degli stati e le principali transizioni. Sono state inoltre introdotte le **eccezioni personalizzate**, utilizzate per segnalare situazioni come cliente minorenne, patente assente o scaduta, capacità passeggeri superata, prenotazione sovrapposta, noleggio non avviabile e conflitti durante la manutenzione.

Il **NoleggioBarcheController** è stato realizzato come punto centrale della logica applicativa. La GUI richiama i metodi del Controller e non accede direttamente ai file di persistenza. Nel Controller sono state concentrate le operazioni relative a clienti, sedi, barche, prenotazioni, noleggi e manutenzioni, insieme ai controlli necessari per applicare le regole del dominio prima di salvare le modifiche.

Per la persistenza sono state definite le **interfacce DAO** e le relative implementazioni basate su file CSV. Le operazioni comuni sono state organizzate attraverso un DAO generico e una classe astratta condivisa, mentre le implementazioni specifiche gestiscono clienti, sedi, barche, prenotazioni, noleggi e manutenzioni. I dati vengono quindi salvati in file CSV separati, mantenendo i riferimenti tra gli oggetti tramite identificativi e matricole.

In questa fase sono stati predisposti anche i **dati dimostrativi**, utili per avviare l'applicazione con clienti, sedi, barche, prenotazioni, noleggi e manutenzioni già presenti. I dati demo permettono di rappresentare situazioni differenti, ad esempio barche disponibili, noleggiate, in manutenzione o fuori servizio, oltre a clienti con e senza patente nautica.

Parallelamente è stata sviluppata una **prima versione dell'interfaccia grafica** in Java Swing. L'applicazione prevede un percorso per il Cliente e uno per l'Amministratore. Sono state realizzate le schermate di avvio, login, registrazione, home cliente, catalogo, prenotazione, home amministratore, gestione del noleggio, visualizzazione delle barche e gestione della manutenzione. Le schermate sono state collegate al Controller, mantenendo la GUI concentrata sulla raccolta degli input, sulla navigazione e sulla visualizzazione dei risultati o degli errori.

Sono state quindi rese operative le **funzionalità principali**: autenticazione e registrazione del cliente, consultazione e filtro del catalogo, creazione delle prenotazioni, avvio e conclusione dei noleggi e gestione delle manutenzioni. La logica applicativa controlla **maggiore età, patente, capienza, disponibilità della barca e sovrapposizioni** prima di completare le operazioni richieste.

Per descrivere il comportamento dinamico dei casi d'uso più importanti sono stati inoltre predisposti i **Sequence Diagram** relativi al login, alla creazione della prenotazione, alla gestione del noleggio e alla gestione della manutenzione. Questi diagrammi mostrano il passaggio delle richieste tra utente, GUI, Controller, DAO e file CSV e rendono evidente la separazione delle responsabilità adottata nel progetto.

Infine è stata predisposta la **documentazione Javadoc** delle classi e dei metodi più significativi. I commenti sono stati mantenuti semplici e utili, in modo da descrivere le responsabilità principali senza appesantire il codice. Al termine del secondo homework il progetto disponeva quindi di un modello completo, di un Controller collegato alla persistenza, di una prima GUI funzionante e di una struttura coerente con l'architettura prevista.

Completamento finale del progetto e della documentazione

Nel corso del terzo homework il lavoro si è concentrato esclusivamente sul **completamento finale del progetto** e sulla preparazione della documentazione necessaria alla consegna. Partendo dalla prima versione realizzata nel secondo homework, l'obiettivo è stato quello di rendere l'applicazione pronta per essere presentata e utilizzata nella sua forma definitiva.

Una delle attività principali ha riguardato il **completamento dell'interfaccia grafica**. Le schermate già predisposte sono state rifinite e coordinate tra loro, rendendo più chiari i passaggi tra Start, Login, Registrazione, Home Cliente, Catalogo, Prenotazione, Home Amministratore, Gestione Noleggio e Gestione Manutenzione. In questo modo la GUI è passata da una prima versione funzionante a una versione finale più coerente e ordinata.

Parallelamente è stata completata la **documentazione finale del progetto**, raccogliendo in modo organico le informazioni necessarie per descrivere il sistema, la struttura applicativa e il funzionamento delle principali operazioni. È stato inoltre predisposto il **manuale d'uso**, pensato per spiegare all'utente come avviare il programma, effettuare l'accesso e utilizzare le funzionalità disponibili nei percorsi Cliente e Amministratore.

A supporto della consegna è stata infine preparata una **relazione riepilogativa** delle attività svolte nei tre homework, così da mostrare in modo chiaro l'evoluzione del progetto: dalla progettazione iniziale del dominio, alla prima versione applicativa, fino al completamento dell'interfaccia grafica e della documentazione. Il terzo homework rappresenta quindi la fase conclusiva del lavoro, nella quale il progetto viene organizzato e presentato nella sua versione definitiva.